

02 BFS

Lab 2a: Breadth-First Search

Given this tree, find a path from A to H.

A → B, C ; B → D, E ; C → F, G ; E → H

(D, F, G, H are levels)

write a program that returns the path as a list such as ['A', 'B', 'E', 'H'] and also returns the nodes in the order they were visited.

Answer this: BFS and DFS differ by exactly one line. Which line and why does changing it produce a completely different algorithm?

⇒ What is BFS? — BFS (Breadth First Search) is a way to explore a tree or graph level by level.

Think of it like standing at the entrance of a building —

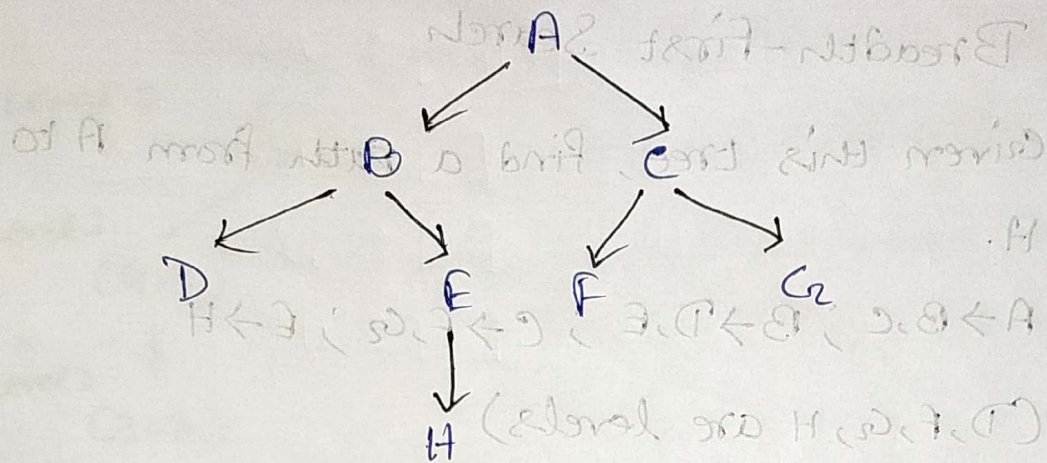
i) 1st you visit all rooms on floor 1.

ii) Then " " " " " " " " 2.

iii) " " " " " " " " 3.

So, BFS always explore the nearest node first.

● Given Tree : —



Connections $\Rightarrow$ $A \rightarrow B, C$
 $B \rightarrow D, E$
 $C \rightarrow F, G$
 $E \rightarrow H$

Goal $\Rightarrow$ Find a path from A to H

● Step-by-Step BFS : —

i) Start $\Rightarrow$ Queue = [A]

visited order = []

ii) Visit A $\Rightarrow$ Visited = [A]

Add B and C to queue.

Queue = [B, C]

iii) Visit B $\Rightarrow$ Visited = [A, B]

Add D and E to queue.

Queue = [C, D, E]

iv) Visit C $\Rightarrow$ Visited = [A, B, C]

Add F and G to queue.

Queue = [D, E, F, G]

i) visit D $\Rightarrow$ visited = [A, B, C, D]
Queue = [E, F, G]

ii) visit E $\Rightarrow$ visited = [A, B, C, D, E]
Add H.
Queue = [F, G, H]

iii) visit F $\Rightarrow$ visited = [A, B, C, D, E, F]
Queue = [G, H]

iv) visit G $\Rightarrow$ visited = [A, B, C, D, E, F, G]
Queue = [H]

v) visit H $\Rightarrow$ visited = [A, B, C, D, E, F, G, H]

Goal Found!

● Path Found : — A $\rightarrow$ B $\rightarrow$ E $\rightarrow$ H

Path list $\Rightarrow$ ['A', 'B', 'E', 'H']

visited order $\Rightarrow$ ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']

● Python Program :

1. from collections import deque

2. graph = {

3. 'A' : ['B', 'C'],

4. 'B' : ['D', 'E'],

5. 'C' : ['F', 'G'],

6. 'D': []
7. 'E': ['H'],
8. 'F': []
9. 'G': []
10. 'H': []
11. 3

12. queue = deque([['A', ['A']]])
13. visited = []

14. while queue:

15. node, path = queue.popleft()

16. if node not in visited:

17. visited.append(node)

18. if node == 'H':

19. print("Path:", path)

20. print("Visited:", visited)

21. break

22. for neighbor in graph[node]:

23. queue.append((neighbor, path + [neighbor]))

Output ⇒ Path: ['A', 'B', 'E', 'H']
 visited: ['A', 'B', 'E', 'D', 'E', 'F', 'G', 'H']

code Explanation :

1. i) Import deque
ii) It helps us make a queue easily.
2. create a graph (tree)
3. A is connected to B and C.
4. B is connected to D and E.
5. C is connected to F.
6. D has no children.
7. E is connected to H.
8. 9. 10. F, G, and H have no children.
11. End the graph.
12. i) create a queue
ii) start from A
iii) current path is ['A']
iv) Queue contains [('A', ['A'])]
13. i) Empty list
ii) store visited nodes
14. Run while queue is not empty.
15. i) Take the 1st item from queue.
ii) store node in node.
iii) store path in path.
Eg $\xrightarrow{\quad} ('A', ['A'])$ becomes
node = 'A' ; path = ['A']

16. Check if node was visited before,

17. Add node to visited list.

Eg visited = ['A']

18. Check if we found H.

19. Print the Path

Eg output = ['A', 'B', 'E', 'H']

20. Print visited nodes

Eg output $\Rightarrow$ visited: ['A', 'B', 'C', 'D', 'E', 'F', 'G', 'H']

21. Stop the Program.

22. Look at all children of the current node.

Eg graph ['A']

gives ['B', 'C']

So neighbor becomes: B

[['A'], 'A'] then C

23. i) Add child to queue.

ii) Also create a new path

Eg Current: Path = ['A']

neighbor = 'B'

Then: Path + [neighbor]

Becomes: ['A', 'B']

Added to queue: ['B', ['A', 'B']]

Summary $\Rightarrow$ i) BFS takes one node from the front of the queue.

ii) visits it

iii) Add its children to the back of the queue

iv) keeps doing this until it finds H.

• BFS vs DFS :

BFS $\Rightarrow$ node, Path = queue.popleft()

Removes from the front.

Result: A, B, C, D, E, F, G, H

Level by level

DFS $\Rightarrow$ node, Path = queue.pop()

Removes from the end.

Result: A, C, G, F, B, E, H

Goes deep first

• Why Does One Line change Everything? : —

BFS $\Rightarrow$ popleft()

Takes the oldest node first.

Like standing in a queue.

A

B C

D E F G

H

visits level by level.

* popleft() uses
a queue (FIFO)

DFS $\Rightarrow$ POP()

Takes the newest node first.

Like a stack of books.

You always take the top book first.

So, it keeps going deeper before coming back

* POP() uses a stack (LIFO):

DFS $\Rightarrow$ queue, path = queue, pop()

Removes from the front.

Result: A, B, C, D, E, F, G, H

Level by level

DFS $\Rightarrow$ queue, path = queue, pop()

Removes from the end.

Result: A, C, G, F, B, E, H

Goes deep first

Prints out the path